AI Audit Report

Student ID: 23127081

Full name: Nguyễn Phan Hùng Linh

1. Declaration

I use AI tools for the following tasks. I used a second Codex/GPT-5.6 agent to help with test design and raw-result analysis. I acted as the human reviewer. I made the final scope, parameter, execution, and reporting decisions.

The AI did not create the final JTL evidence. The measured files came from the JMeter runs on my computer.

2. Interaction 1 — Test design

Field	Record
AI tool	Codex / GPT-5.6
Date and time	2026-08-16 ICT
Purpose	Get a first design for Load, Stress, and Spike tests

2.1 My prompt

> Read HW05.md and inspect the eshop-sut project read-only. Recommend a simple, compliant plan: deliverables, non-conflicting endpoints for Load/Stress/Spike, test scenarios/metrics, and likely tools. Give concise advice and leave an audit-ready account of your consultation. Do not edit files.

2.2 AI output

The AI proposed this first mapping:

Test	Suggested endpoint	Suggested CSV
Load	`GET /api/orders/{order_id}`	`data/order_ids.csv`
Stress	`POST /api/register`	`data/register_users.csv`
Spike	`POST /api/apply-coupon`	`data/coupons.csv`

It suggested 20 users, a 60-second ramp-up, and a five-minute steady period for Load. It suggested increasing Stress from 10 to 50 users. For Spike, it suggested 2 → 40 → 2 users.

It also suggested Summary Report, Aggregate Report, and View Results Tree as three different listener types. It recommended HTTP and JSON assertions and warned that View Results Tree may use too much memory in a long test.

2.3 My review and final decision

The proposed endpoints covered the three required endpoint groups. I chose a broader workflow with three related endpoints in each scenario. This was my scope choice, not an HW05 requirement.

I made these changes:

- used a separate CSV file for every scenario;
- added status assertions and required JSON extraction;
- reduced the first measured Stress test from 50 users to 30 users;
- used smoke tests before the measured runs; and
- kept View Results Tree only in the short Spike plan.

3. Interaction 2 — Result analysis

Field	Record
AI tool	Codex / GPT-5.6
Date and time	2026-08-16 ICT
Purpose	Analyse JTL and resource files and suggest thresholds

3.1 My prompt

> Read the raw JTL files in results/load.jtl, results/stress.jtl, results/spike.jtl, and results/endurance.jtl plus their resource CSVs. Analyze performance, propose thresholds and optimizations. Be concise but show numbers and conclusions. Do not edit files. I will independently check all values.

3.2 AI output

The AI produced this result table:

Run	Samples	RPS	Errors	Mean	p95	p99	Maximum
Load	5,348	17.90	0.00%	8.56 ms	30 ms	68 ms	178 ms
Stress	8,522	47.55	0.00%	25.06 ms	139 ms	339 ms	648 ms
Spike	3,711	18.77	0.00%	22.05 ms	97 ms	403 ms	622 ms
Endurance	55,622	92.89	0.00%	19.04 ms	83 ms	293 ms	1,031 ms

It reported these backend resource values:

Run	Average CPU	Peak CPU	Average RSS	Peak RSS
Load	1.99%	17.3%	33,530 KB	45,744 KB
Stress	9.17%	24.6%	42,382 KB	64,304 KB
Spike	3.72%	18.0%	34,357 KB	63,664 KB
Endurance	7.76%	26.9%	57,294 KB	76,000 KB

The AI suggested CI gates of error rate at most 1%, endurance p95 at most 150 ms, p99 at most 600 ms, and throughput at least 85 RPS. It suggested database indexes and SQLite WAL as possible improvements. It said a normal connection pool was unsuitable for this local SQLite SUT and that caching was not supported by the current evidence.

3.3 My verification and correction

I recalculated the important values from the JTL elapsed column. The Load, Stress, and Endurance percentiles agreed with the AI output.

I found two Spike percentile differences:

- the AI reported 97 ms for the merged Spike p95; my raw-data check gave **93 ms**; and
- the AI reported 105 ms for the high-load stage p95; my raw-data check gave **103 ms**.

I also report endurance throughput as **92.70 RPS** because I divide 55,622 samples by the planned 600 seconds. The AI used the observed JTL time window and calculated 92.89 RPS. This is a denominator difference. Neither number is proof of the absolute hardware limit.

4. Human responsibility

I verified that resource monitoring followed the backend process `/opt/homebrew/bin/node server.js`. I kept the raw evidence, checked important metrics, and reported only measured performance results. I take responsibility for the final plans, results, and conclusions.